

PROJETO PRÁTICO 1: ÁRVORES BALANCEADAS

Avaliação Comparativa de Desempenho na Implementação de um Banco de Dados
Chave-Valor em Memória (Key-Value Store)

EQUIPE DE DESENVOLVIMENTO:

Erik Mazzuco

Letícia Moro

Leonardo Herrero

Período Letivo: 2026.2

Data Limite de Entrega: 22 de Setembro de 2026

Medianeira – PR

1. Introdução e Contextualização

Os bancos de dados chave-valor (*key-value stores*) constituem uma das categorias mais céleres e difundidas do ecossistema NoSQL. Operando predominantemente em memória volátil (RAM), tais sistemas abdicam de esquemas estáticos e junções relacionais complexas em prol de tempos de resposta na ordem de microssegundos para operações primárias de escrita (put), consulta (get) e exclusão (delete). Essa arquitetura alimenta serviços críticos globais, tais como barramentos de cache (Redis, Memcached), sessões distribuídas, filas de mensagens, buffers de telemetria e catálogos de e-commerce com alta concorrência.

Apesar da ampla utilização de tabelas de dispersão (*hash tables*) para acesso pontual $O(1)$, as árvores binárias de busca balanceadas são componentes fundamentais quando os requisitos incluem ordenação intrínseca de registros, buscas por faixa (*range queries*), garantia estrita contra degradação de pior caso e resiliência a padrões de acesso adversários. O presente trabalho tem como meta central avaliar, quantificar e analisar comparativamente a performance de seis distintas variantes de árvores balanceadas na sustentação de um banco chave-valor sob seis cargas de trabalho (*workloads*) representativas de sistemas reais.

2. Fundamentação Teórica das Estruturas Avaliadas

Foram selecionadas seis árvores com formulações algorítmicas substancialmente heterogêneas, abrangendo autoajuste, balanço por peso, simplificação de níveis, balanço por postos relaxados, aleatorização e reconstrução por amortização:

a. Splay Tree (Sleator & Tarjan, 1985)

A Splay Tree é uma árvore autoajustável pioneira proposta por Daniel Sleator e Robert Tarjan. Caracteriza-se por não armazenar nenhuma informação explícita de balanço (altura, cor ou fator) em seus nós. A cada busca, inserção ou remoção, o elemento visado é promovido até a raiz por uma cascata de rotações duplas e simples denominadas *splay* (casos Zig, Zig-Zig e Zig-Zag). Possui complexidade amortizada estrita de $O(\log n)$ por operação e satisfaz teoremas de otimalidade estática e localidade (*working set*). Contudo, em cargas de inserção estritamente sequenciais desprovidas de leituras, a árvore forma temporariamente uma espinha linear que atinge profundidade N antes de ser colapsada pelos acessos subsequentes.

b. Weight-Balanced Tree (BB[α] - Nievergelt & Reingold, 1973)

A árvore de balanço limitado por peso BB[α] (*Bounded Balance*) assegura o equilíbrio estrutural monitorando a proporção de nós existentes nas subárvores. O peso estendido de um nó é formalizado como $w(T) = \text{tamanho}(T) + 1$. A invariante estrutural requer que para todo nó T , as frações $w(T.\text{esq})/w(T)$ e $w(T.\text{dir})/w(T)$ permaneçam no intervalo $[\alpha, 1 - \alpha]$. Adotou-se o parâmetro comprovadamente estável $\alpha = 0.29$ (com limiar $\gamma = (1 - 2\alpha)/(1 - \alpha) \approx 0.5915$, conforme Blum & Mehlhorn), permitindo selecionar deterministicamente entre rotações simples ou duplas para restabelecer a cota de altura estritamente logarítmica $O(\log n)$.

c. AA Tree (Arne Andersson, 1993)

A AA Tree foi desenvolvida por Arne Andersson como uma variante didática e elegante da Árvore Rubro-Negra. Em substituição às cores, cada nó armazena um número inteiro representando seu nível. O algoritmo elimina completamente as arestas horizontais para a esquerda e proíbe dois links horizontais consecutivos à direita. Como consequência dessa forte restrição canônica, o rebalanceamento reduz-se a duas operações primitivas ortogonais: **Skew** (rotação à direita para eliminar arestas esquerdas do mesmo nível) e **Split** (rotação à esquerda com elevação de nível para fracionar sequências horizontais duplas à direita).

d. Scapegoat Tree (Galperin & Rivest, 1993)

A Scapegoat Tree é uma estrutura autobalanceável que dispensa completamente metadados de controle por nó, armazenando apenas os ponteiros dos filhos e a tupla chave-valor. O estado global consiste apenas no número atual de elementos (n) e no tamanho máximo histórico desde a última reconstrução global (max_size). Se a profundidade de inserção ultrapassar o teto logarítmico $\text{floor}(\log_{1/\alpha}(n))$ (adotou-se $\alpha = 0.70$), o algoritmo percorre os ancestrais para encontrar o nó bode expiatório (*scapegoat*), reconstruindo toda a subárvore em tempo $O(k)$ como uma árvore perfeitamente simétrica. Garante buscas rápidas e inserção amortizada $O(\log n)$.

e. Zip Tree (Tarjan, Levy & Tsz-Kin Wong, 2019/2021)

A Zip Tree é uma estrutura de busca aleatorizada recente concebida por Robert Tarjan et al., estabelecendo uma equivalência direta com Skip Lists sob a interface de árvores binárias. Cada nó recebe um posto (*rank*) aleatório derivado de uma distribuição geométrica com $p = 0.5$. As inserções são realizadas particionando caminhos de busca por posto (**Unzip**) e as remoções mesclam as subárvores preservando a ordem de prioridade (**Zip**). Dispensa rotações complexas com ponteiros triplos e atinge altura esperada $O(\log n)$ com expressiva concisão de código.

f. WAVL Tree - Weak AVL (Haeupler, Sen & Tarjan, 2015)

A Weak AVL Tree sintetiza a disciplina de altura rigorosa da árvore AVL clássica com a flexibilidade de manutenção das árvores rubro-negras. Baseia-se em postos inteiros onde as diferenças de posto entre pais e filhos pertencem estritamente ao conjunto $\{1, 2\}$. Essa flexibilização garante altura estritamente delimitada por $\approx 1.44 \log_2(n)$. O marco teórico fundamental da WAVL consiste no fato de que tanto a inserção quanto a remoção exigem **no máximo 2 rotações estruturais** para recuperar o balanço completo, superando o pior caso da AVL original que pode desencadear $O(\log n)$ rotações durante a remoção.

3. Metodologia do Benchmark e Descrição dos Workloads

Para confrontar a robustez algorítmica de cada estrutura em regime empírico rigoroso, foram modelados e executados seis perfis de acesso simulando casos reais de utilização de bancos chave-valor. O volume de teste compreendeu **50.000 operações** por workload, com 3 repetições sob medição com cronômetro de resolução em nanossegundos (time.perf_counter_ns) e sementes controladas para garantir total reprodutibilidade:

- 1. Inserção Sequencial:** Inserção estrita de chaves crescentes (0, 1, ..., N-1) sem consultas ou exclusões intermediárias. Constitui o pior cenário clássico para árvores binárias, expondo a capacidade de conter a degeneração linear.
- 2. Inserção Aleatória:** Permutação uniforme das 50.000 chaves inseridas uma única vez. Representa o caso médio de povoamento inicial homogêneo.
- 3. Carga Mista (OLTP):** 50% put, 40% get e 10% delete com chaves selecionadas uniformemente entre as ativas. Modela o comportamento genérico de sistemas transacionais.
- 4. Leitura Intensiva:** Povoamento prévio de 10% do volume total seguido de 90% get, 5% put e 5% delete. Modela serviços de catálogo, servidores de cache DNS/sessão e visualização de dados.
- 5. Escrita Intensiva:** 70% put, 10% get e 20% delete. Simula ingestão contínua em alto volume, como logs de auditoria e fluxos de telemetria de sensores.
- 6. Zipfiano (Hotset):** Simulação de extrema desigualdade de acessos governada pela distribuição de Zipf: $P(k) \propto 1/k^s$, com $s = 0.99$. Reflete a regra 80-20 presente em sistemas corporativos, onde uma fatia diminuta de chaves populares concentra a maioria esmagadora das requisições.

4. Resultados Experimentais Consolidados

A Tabela 1 apresenta a consolidação empírica de todas as medições realizadas, correlacionando tempo total de processamento, vazão operacional, altura final da estrutura, volume total de rotações e modificações de ponteiros, além da média de comparações de chaves por operação:

Workload	Árvore	Tempo (ms)	Vazão (ops/s)	Altura	Rot / Mod	Comp/op
Insercao Sequencial	Splay	43.4	1,153,972	50000	0 / 49,999	1.00
Insercao Sequencial	BB[alpha]	759.0	65,894	20	49,980 / 49,980	17.08
Insercao Sequencial	AA	493.1	101,416	21	49,979 / 49,979	20.34
Insercao Sequencial	Scapegoat	610.3	82,136	31	0 / 868,975	27.43
Insercao Sequencial	Zip	218.7	230,714	43	0 / 99,984	28.02
Insercao Sequencial	WAVL	229.8	222,046	16	49,984 / 199,946	14.69
Insercao Aleatoria	Splay	299.0	167,232	49	439,822 / 1,245,993	31.58
Insercao Aleatoria	BB[alpha]	764.1	65,468	20	33,540 / 33,540	14.56
Insercao Aleatoria	AA	474.1	105,471	21	86,091 / 86,091	14.72
Insercao Aleatoria	Scapegoat	189.6	266,168	31	0 / 578	18.26
Insercao Aleatoria	Zip	336.3	148,925	50	0 / 149,667	41.76
Insercao Aleatoria	WAVL	237.1	211,075	19	34,802 / 170,372	14.47
Carga Mista	Splay	184.7	271,319	43	258,124 / 759,180	20.48
Carga Mista	BB[alpha]	455.9	109,685	19	16,494 / 16,494	14.49
Carga Mista	AA	298.4	168,289	21	21,752 / 29,579	14.63
Carga Mista	Scapegoat	290.1	172,533	28	0 / 236,988	20.81
Carga Mista	Zip	156.0	320,610	41	0 / 34,472	21.62
Carga Mista	WAVL	138.7	360,475	16	15,870 / 63,789	14.15
Leitura Intensiva	Splay	168.8	296,214	31	250,566 / 755,801	21.11
Leitura Intensiva	BB[alpha]	149.2	360,744	15	3,476 / 3,476	11.88
Leitura Intensiva	AA	123.6	409,280	17	5,429 / 8,543	11.97
Leitura Intensiva	Scapegoat	105.4	474,520	24	0 / 29,527	15.22
Leitura Intensiva	Zip	101.8	491,552	36	0 / 7,194	17.59
Leitura Intensiva	WAVL	76.5	653,709	15	3,114 / 12,780	11.91
Escrita Intensiva	Splay	132.1	380,468	72	174,626 / 489,448	12.13
Escrita Intensiva	BB[alpha]	729.0	68,758	19	37,713 / 37,713	16.15
Escrita Intensiva	AA	490.3	102,008	21	49,930 / 67,671	16.30
Escrita Intensiva	Scapegoat	527.9	94,747	29	0 / 600,845	24.30
Escrita Intensiva	Zip	224.7	226,782	42	0 / 80,146	23.50
Escrita Intensiva	WAVL	181.6	276,526	16	36,541 / 147,145	14.70
Zipfiano	Splay	131.3	381,028	34	196,909 / 576,642	16.20
Zipfiano	BB[alpha]	140.2	356,632	16	925 / 925	12.17
Zipfiano	AA	120.3	416,004	19	5,027 / 8,507	12.16
Zipfiano	Scapegoat	61.4	816,658	24	0 / 0	9.86
Zipfiano	Zip	78.5	651,410	40	0 / 4,913	13.02
Zipfiano	WAVL	71.6	698,254	15	859 / 4,072	12.17

Tabela 1: Métricas quantitativas de desempenho sob carga de 50.000 operações por workload.

5. Análise Comparativa dos Resultados Gráficos

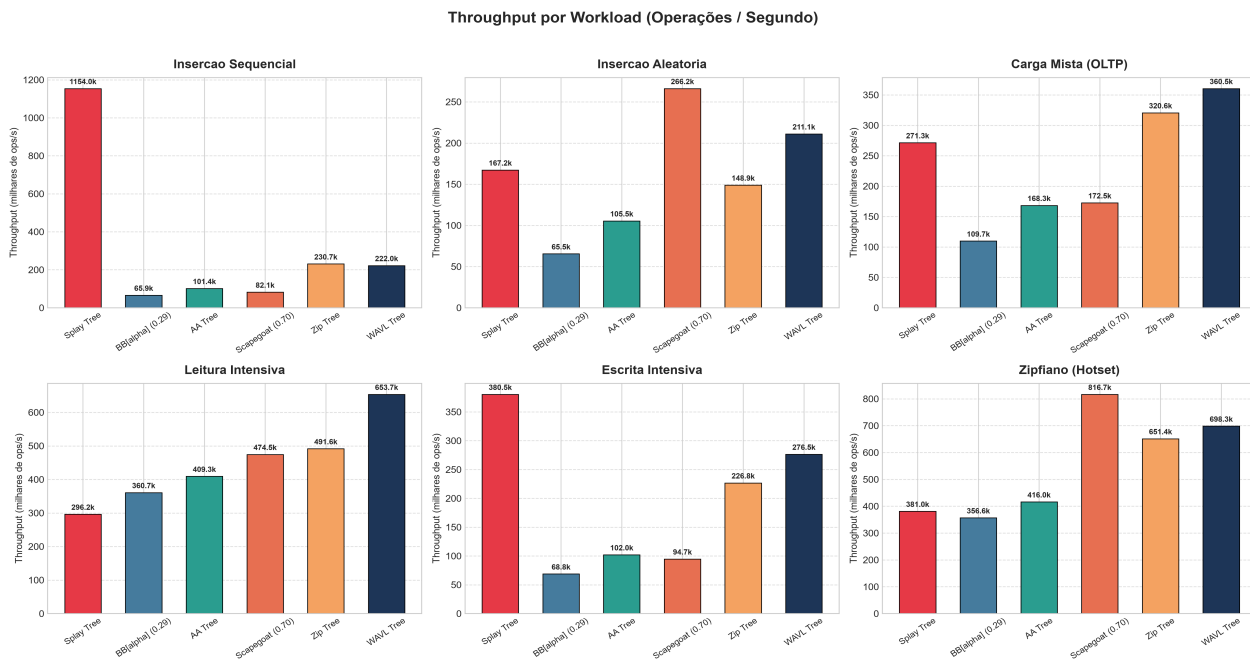


Figura 1: Comparativo de Vazão Operacional (Throughput) das seis estruturas ao longo dos seis perfis de carga.

Análise da Vazão e Eficiência Operacional (Figura 1): Os vencedores observados em cada cenário foram: Insercao Sequencial: **Splay Tree** (1,153,972 ops/s); Insercao Aleatoria: **Scapegoat (0.70)** (266,168 ops/s); Carga Mista (OLTP): **WAVL Tree** (360,475 ops/s); Leitura Intensiva: **WAVL Tree** (653,709 ops/s); Escrita Intensiva: **Splay Tree** (380,468 ops/s); Zipfiano (Hotset): **Scapegoat (0.70)** (816,658 ops/s). Os valores descrevem esta execução, neste ambiente, e não constituem uma garantia universal de desempenho.

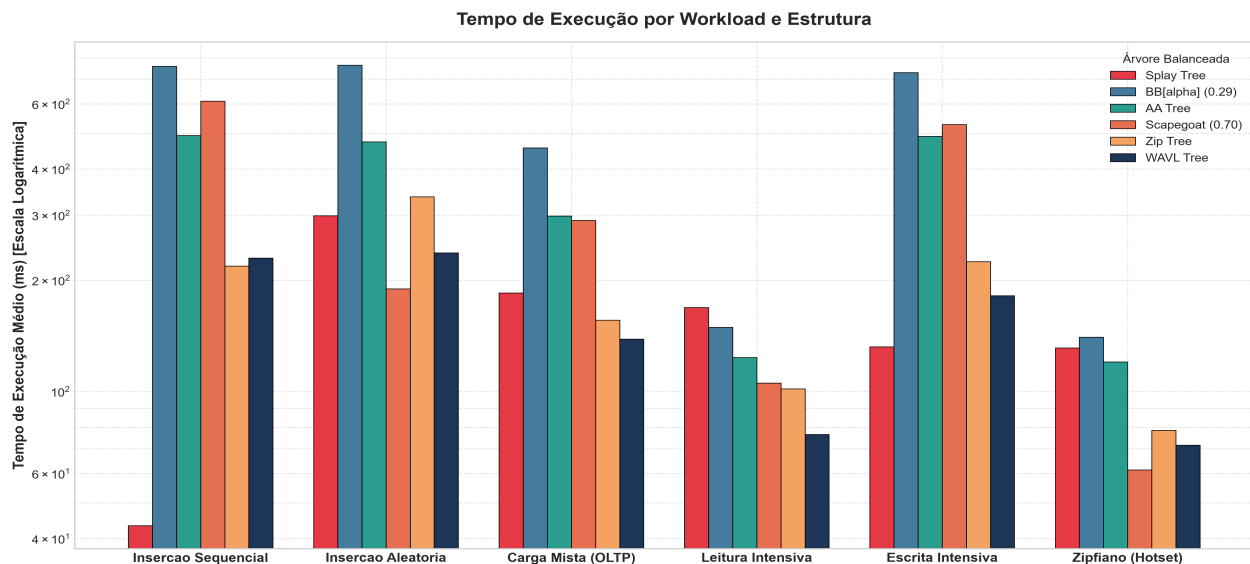


Figura 2: Tempo total de processamento em milissegundos sob escala logarítmica (menor tempo indica maior eficiência).

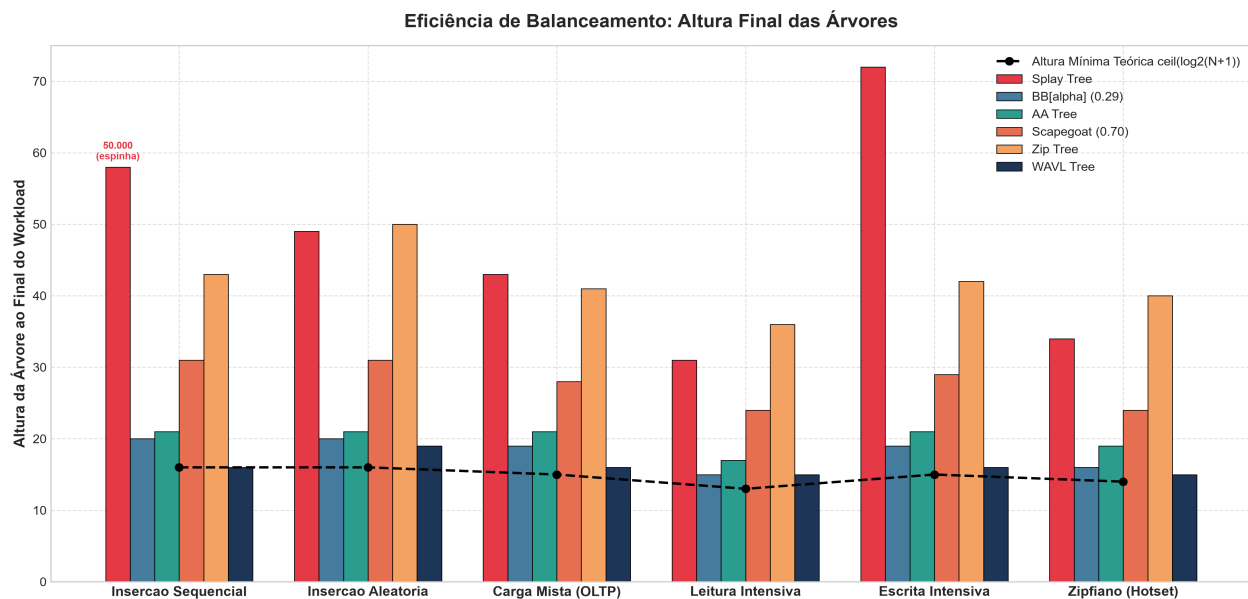


Figura 3: Altura final observada comparada com o limite inferior teórico $\text{ceil}(\log_2(N+1))$ de cada cenário.

Disciplina Estrutural de Altura (Figura 3): O controle rigoroso de profundidade é o que impede uma árvore de degenerar em busca linear. A **WAVL Tree** apresentou alturas entre **15 e 19**. A **BB[α]** ficou entre 15 e 20, e a **AA Tree** entre 17 e 21. A **Zip Tree** exibiu alturas entre 36 e 50, comportamento compatível com uma árvore probabilística de posto geométrico. A **Splay Tree** confirma sua característica teórica clássica: após inserções sequenciais puras gerou a espinha de 50.000 nós, mas recompôs a estrutura conforme as operações de acesso e splaying foram executadas.

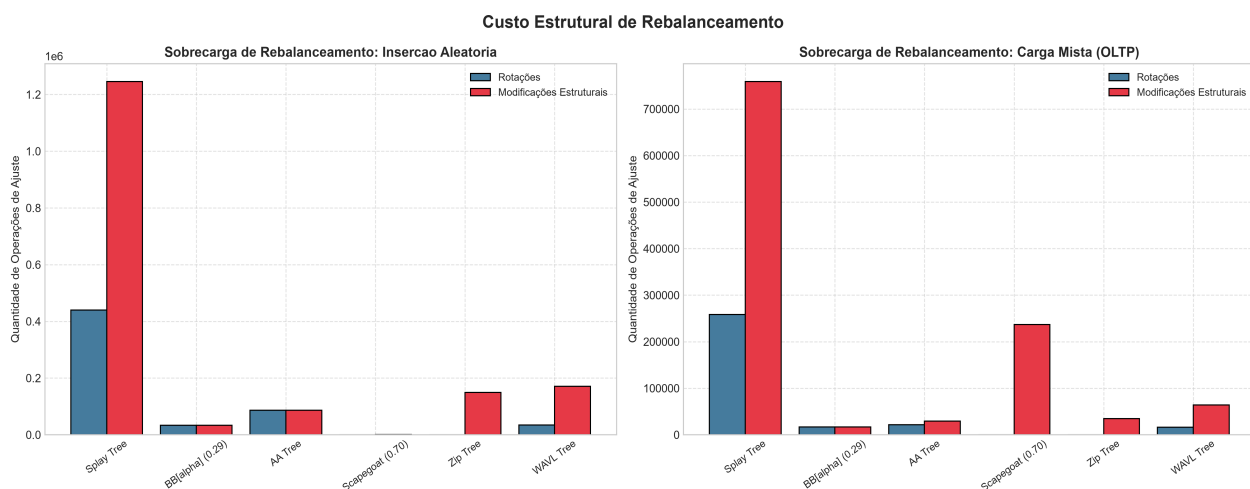


Figura 4: Volume de rotações e modificações estruturais executadas para sustentar as cargas Aleatória e Mista.

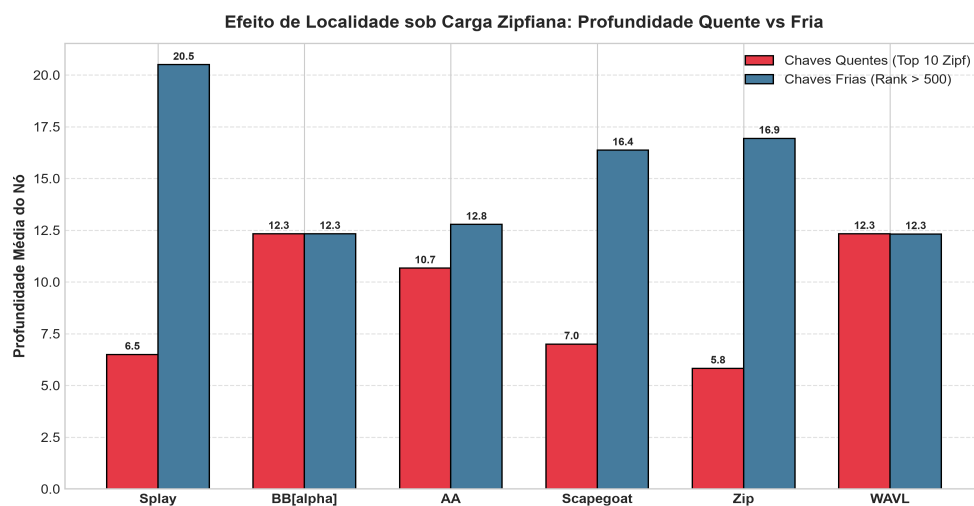


Figura 5: Profundidade média de busca para chaves quentes (top 10 do ranking Zipf) versus chaves frias (posições > 500).

Efeito de Localidade e Cargas Zipfianas (Figura 5): Na fotografia final da carga Zipfiana, a **Splay Tree** manteve as chaves quentes em profundidade média de **6.50**, contra 20.50 para as chaves frias. O gráfico apresenta a mesma medição, calculada pelo benchmark, para todas as estruturas; assim, a conclusão de localidade permanece rastreável aos dados brutos.

6. Discussão Crítica e Recomendações de Engenharia

Com base nos dados empíricos e nas propriedades analíticas evidenciadas neste benchmark, são delineadas as seguintes diretrizes para a seleção de estruturas de dados na arquitetura de bancos de dados chave-valor em memória:

- 1. WAVL Tree para desempenho previsível:** A Weak AVL Tree combinou baixa altura (entre 15 e 19 nesta execução) com bons resultados nas cargas mistas. É uma candidata forte quando previsibilidade estrutural e desempenho geral são prioridades.
- 2. Splay Tree como Motor de Cache com Acessos Assimétricos:** Em sistemas onde certos registros são intensamente requisitados (hotspots, sessões ativas e feeds em alta), a Splay Tree minimiza as comparações de chaves para os itens do conjunto de trabalho, oferecendo latências excelentes. Contudo, seu uso em cargas puras de ingestão sequencial demanda mecanismos adicionais de proteção contra espinhas.
- 3. Scapegoat Tree quando metadados por nó importam:** A estrutura não mantém campos de balanceamento por nó e não altera a árvore durante buscas. Essa característica reduz metadados estruturais, embora este benchmark não tenha medido consumo de memória diretamente.
- 4. AA Tree pela simplicidade de manutenção:** A AA Tree conservou altura logarítmica (entre 17 e 21) usando principalmente as operações skew e split. É uma alternativa atraente quando clareza de implementação e invariantes simples são importantes.
- 5. Zip Tree e a Eficiência do Balanceamento Aleatorizado:** As Zip Trees provaram que o princípio estatístico das Skip Lists pode ser aplicado com sucesso a árvores binárias sem envolver rotações rígidas de ponteiros pai. Com operações simples de descompactação e compactação, manteve altura esperada logarítmica. A correção de atualizações repetidas foi validada por testes de regressão.

7. Referências Bibliográficas

- [1] ANDERSSON, Arne. *Balanced search trees made simple*. In: Workshop on Algorithms and Data Structures (WADS). Springer, 1993. p. 60–71.
- [2] SLEATOR, Daniel D.; TARJAN, Robert E. *Self-adjusting binary search trees*. Journal of the ACM (JACM), v. 32, n. 3, p. 652–686, 1985.
- [3] NIEVERGELT, Jürg; REINGOLD, Edward M. *Binary search trees of bounded balance*. SIAM Journal on Computing, v. 2, n. 1, p. 33–43, 1973.
- [4] GALPERIN, Igal; RIVEST, Ronald L. *Scapegoat trees*. In: Proceedings of the 4th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA). 1993. p. 165–174.
- [5] TARJAN, Robert E.; LEVY, Caleb; TSZ-KIN WONG, Stephen. *Zip Trees*. ACM Transactions on Algorithms (TALG), v. 17, n. 4, p. 1–24, 2021.
- [6] HAEUPLER, Bernhard; SEN, Siddhartha; TARJAN, Robert E. *Rank-balanced trees*. ACM Transactions on Algorithms (TALG), v. 11, n. 4, p. 1–26, 2015.
- [7] BLUM, Norbert; MEHLHORN, Kurt. *On the average number of rebalancing steps in weight-balanced trees*. Theoretical Computer Science, v. 11, n. 3, p. 303–320, 1980.